

O objetivo é proceder à representação de uma Picture num espaço latente de N dimensões.

$$\mathcal{P}_t = f(S_t^{UAV}, S_t^{UGV}, S_t^{RANTL}, S_t^{TOS}) \in R^d$$

Em que cada S_t^X corresponde ao estado atual de cada uma das fontes de dados. Sendo que os estados S_t^X já são resultado de encoding de dados multimodais, uma vez que os UAVs e UGVs fazem recolha de diferentes dados, por exemplo: **imagem, sensores e informação de rede.**

O que é que cada agente vai transmitir?

Para se produzir o embedding de dados multimodais deve haver uma função de fusão capaz de juntar os embeddings já feitos das diferentes modalites:

$$\zeta_t = \mathcal{F}(e_1, e_2, \dots, e_n)$$

Em que cada e_k corresponde ao embedding feito localmente, em que cada k corresponde a uma fonte diferente de dados (**sensor IMU, GPS data, imagem da câmara, informação de users servidos**).

A função de fusão $\mathcal{F}$, feita através de um multimodal transformer, deve processar e fundir os vários embeddings.

Proposta de otimização

Antes dos embeddings/tokens serem transmitidos poderão ser aplicados alguns métodos de redução de dimensões ou quantização, ainda a ser estudada a melhor forma (**PCA, int8, gzip...**). Por enquanto esta compressão será representada por

$$\lambda_t = \mathcal{z}(\zeta_t)$$

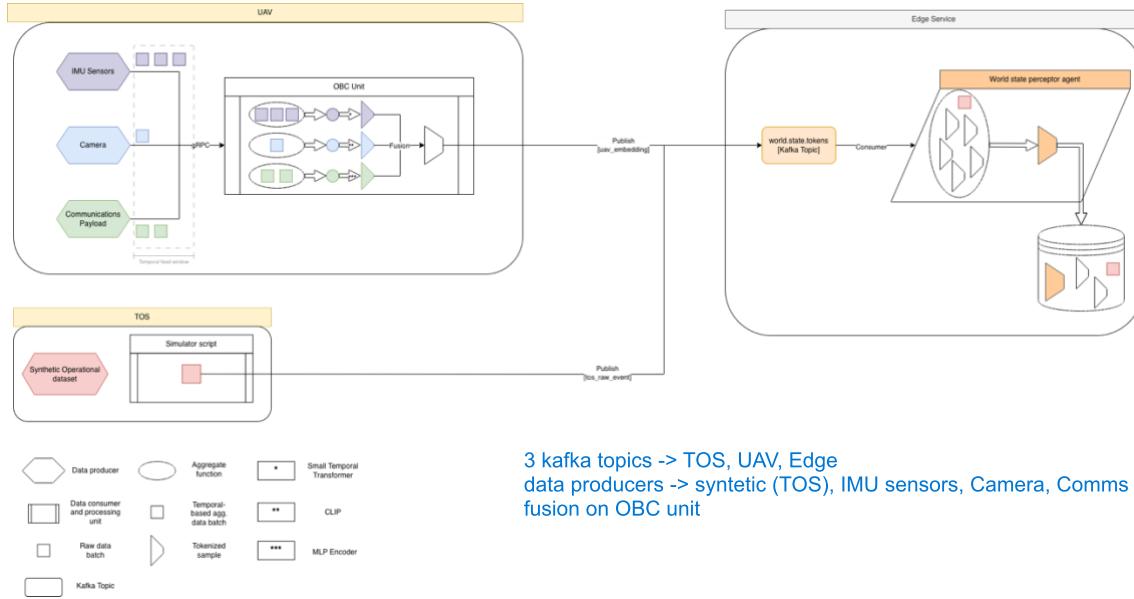
Q: Mas... Como é que o agente que recebe as perceções dos outros vai conseguir decodificar a informação?

A: O agente que recebe os dados vai ter de aplicar a descompressão de forma a ficar com o embedding original gerado pelo agente emissor. De forma a fazê-lo deverá recuperar o embedding gerado na fonte (ζ_t) da seguinte forma:

$$\zeta_t = \mathcal{z}^{-1}(\lambda_t)$$

agente que recebe -> descomprime e fica com o embedding original

Como é realizada a organização em termos de dataflow na arquitetura - *Multimodal event-driven stream-based cognitive architecture?*



Em princípio teremos cada um dos **publishers** (assinalados a amarelo) a **enviar informação para um edge server**. No edge server irão residir os tópicos e o real-time data batch processing em que:

Opção 1 [Atual]: Teremos um tópico geral e o agente que reside no edge server terá a capacidade de processar os dados, agregar e dar a devida significância a cada componente de forma a criar uma Picture ($\mathcal{P}_t$).

Opção 2: Vários tópicos serão deployed no edge server de forma que cada componente de diferentes classes envie para o respetivo tópico a informação, sendo que o edge server pode ter um ou mais agentes a subscrever e a processar os conteúdos do tópico. Numa outra fase separada deverá ser capaz de agregar os dados à mesma e criar uma Picture ($\mathcal{P}_t$).

De momento ainda não está operacional o processamento de dados em tempo real.

Exemplo de criação de embedding para um UAV com dados sintéticos

Para o cenário simulado atual os dados são todos sintéticos e foram gerados através do script `generate_synthetic_data.py`.

- IMU data (`imu_data.json`) => Contém dados do acelerômetro, giroscópio, barômetro e magnetômetro. Os dados foram gerados a partir do standard IEEE Std 952-1997 que corresponde às vibrações de giroscópios e acelerômetros.

timestamp_s	Relative time event generated data [s]
acc_x	X-axis acceleration [m/s ²]
acc_y	Y-axis acceleration [m/s ²]
acc_z	Z-axis acceleration [m/s ²]
gyro_roll	Roll angular rate [rad/s]
gyro_pitch	Pitch angular rate [rad/s]
gyro_yaw	Yaw angular rate [rad/s]
pressure_hPa	Barometric pressure [hPa]
mag_north	Magnetic field north component
mag_east	Magnetic field east component
mag_down	Magnetic field down component
roll_deg	UAV roll angle [°]
pitch_deg	UAV pitch angle [°]
yaw_deg	UAV heading/yaw angle [°]
speed_mps	UAV speed [m/s]
flight_phase	UAV flight stage [takeoff; cruise; loiter; descent]

- Network KPM data (`network_kpm_data.json`) => Coleção de métricas de desempenho da rede para UEs servidos por um UAV a atuar como gNB. O modelo de canal utilizado para gerar os dados foi o 3GPP TR 38.811.

timestamp_s	Relative time event generated data [s]
report_idx	Sequential network report identifier [#]
uav_altitude_m	UAV altitude [m]
flight_phase	UAV flight stage [takeoff; cruise; loiter; descent]
ue_x_m	UE X-axis ground position [m]
ue_y_m	UE Y-axis ground position [m]
horiz_dist_m	Horizontal distance UAV-UE [m]
dist_3d_m	Distance between UAV-UE [m]
elevation_angle_deg	Elevation angle UAV-UE [°]
rsrp_dbm	Reference signal received power [dBm]
rsrq_db	Reference signal received quality [dBm]

sinr_db	Signal-to-interference plus noise ratio [dB]
cqi	Channel quality indicator [0-15]
mcs	Modulation and coding scheme [0-28]
prb_allocated	Allocated physical resource blocks [#]
prb_utilisation_pct	Physical resource blocks usage rate [%]
dl_throughput_mbps	Downlink throughput [Mbps]
ul_throughput_mbps	Uplink throughput [Mbps]
bler	Block error rate [0-1]
latency_ms	UAV-UE air interference latency [ms]

- Synthetic aerial image data (./images & images_metadata.json) => Este dataset é composto por uma pasta de imagens aéreas geradas através dos dados de movimento do UAV, obtidos a partir dos sensores IMU. As imagens têm alguma variabilidade em termos de textura e sombras no terreno, motion blur, etc.

Nota: Nas imagens ainda não se é possível observar os UEs mas como próxima iteração podemos criar um dataset pequeno em lab para o fazer.

UE -> User Equipment

frame_id	Sequential frame identifier [#]
timestamp_s	Relative time event generated data [s]
filename	Image filename pointer
shape	Image resolution shape [H,W,C]
imu_ref_idx	Reference IMU sample index [#]
altitude_m	UAV altitude [m]
speed_mps	UAV speed [m/s]
heading_deg	UAV heading angle [°]
roll_deg	UAV roll angle [°]
pitch_deg	UAV pitch angle [°]
flight_phase	UAV flight stage [takeoff; cruise; loiter; descent]
cam_pan_x	Camera horizontal pan position [px]
cam_pan_y	Camera vertical pan position [px]

Uma representação mais visual dos dados foi gerada e armazenada no ficheiro uav_dashboard.png:



Após estarem gerados os dados criou-se um script para fazer o launch da pipeline gRPC que permite o traffic flow simulado entre os sensores e a obc_unit.

Cada sensor é tratado como um publisher de informação na pipeline gRPC, isto porque, no fundo, também serão os seus dados (agregados) que acabarão por ser streamed para o tópico Kafka presente no Edge Server.

Enquanto os publishers gRPC são scripts muito simples que apenas enviam os dados o on board computer da nossa arquitetura já é implementado através de uma estrutura complexa (obc) que inclui o script (obc_server.py) que recebe os dados da pipeline gRPC e os agrega. De forma a ser funcional a gRPC e desempenhar as funcionalidades expectáveis (agregação + representação num espaço latente) a estrutura obc tem unidades auxiliares que permitem agregar os dados (aggregators) e fazer o processamento (encoding) e transformação (embeddings) da informação.

Sequencialmente as tarefas da OBC são dadas por:

1. Receção dos dados na pipeline gRPC

- a. Armazenamento dos dados consoante a sua fonte num buffer dedicado

2. Processamento contínuo dos dados presentes nos buffers em que os dados principais das diferentes fontes são analisados e postos numa estrutura de dados dedicada (aggregated)

```
aggregated = {
    "timestamp_start": samples[0]["timestamp_s"],
    "timestamp_end": samples[-1]["timestamp_s"],

    "n_samples": len(samples),

    "acc_mean": [
        float(np.mean(acc_x)),
        float(np.mean(acc_y)),
        float(np.mean(acc_z)),
    ],
    "acc_std": [
        float(np.std(acc_x)),
        float(np.std(acc_y)),
        float(np.std(acc_z)),
    ],

    "gyro_mean": [
        float(np.mean(gyro_roll)),
        float(np.mean(gyro_pitch)),
        float(np.mean(gyro_yaw)),
    ],

    "altitude_mean": float(np.mean(altitude)),
    "altitude_max": float(np.max(altitude)),
    "altitude_min": float(np.min(altitude)),

    "dominant_phase": Counter(phases).most_common(1)[0][0]
}
```

Nesta etapa são consideradas as métricas mais relevantes de cada sensores e criam-se representações aritméticas e semânticas da representação do conjunto de dados presente no buffer. Cada fonte de dados tem um script responsável por esta etapa:

IMU – imu_aggregator.py

COMMS – comms_aggregator.py

CAMERA - camera_aggregator.py

3. Após agregados os dados das múltiplas fontes são encoded, sendo que consoante a sua fonte também passam por um processo de encoding distinto, consoante o seu tipo de dados foram escolhidas as seguintes formas de primeira etapa de embedding:
IMU (imu_encoder.py) (128,)=> O IMU encoder pega nos dados agregados na janela do aggregator e utiliza as métricas das amostras recolhidas: média e desvio padrão da aceleração medido pelos acelerómetros, média dos dados provenientes do giroscópio e altitude mínima, máxima e média. Estes dados passam por uma rede neuronal feed-forward (nn.Sequential) composta por duas camadas de ativação produzindo um embedding compacto de 128 dimensões representativo do estado do UAV recolhido pelo IMU.
COMMS (comms_encoder.py) (128,)=> O Comms encoder pega nos dados agregados das métricas de comunicação recolhidas na janela do aggregator e utiliza métricas como SINR médio, máximo e mínimo, throughput médio de downlink, BLER médio e número total de UEs presentes. Estes dados passam por uma rede neuronal feed-forward (nn.Sequential) composta por

duas camadas de ativação produzindo um embedding compacto de 128 dimensões representativo do estado da rede de comunicações entre o UAV e os UEs.

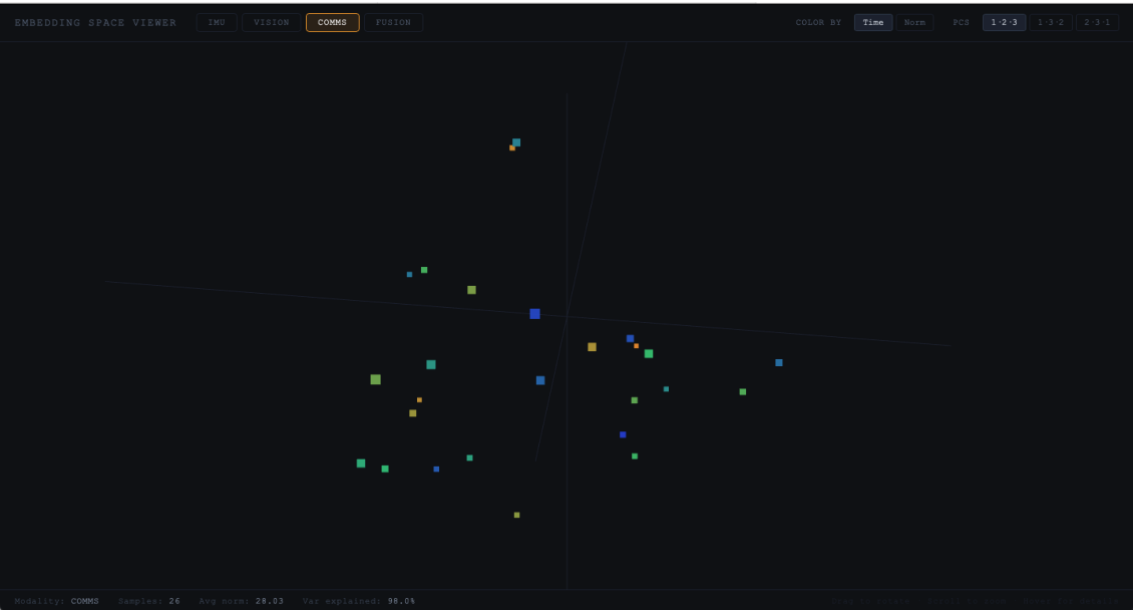
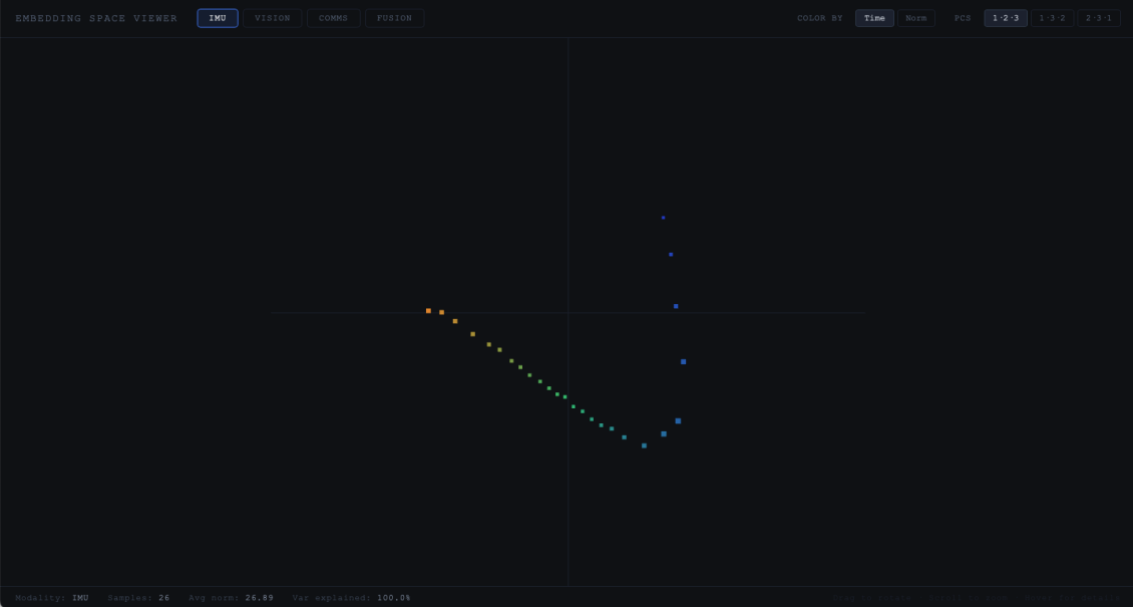
CAMERA (clip_encoder.py) (512,) => O Vision encoder utiliza imagens aéreas recolhidas durante o voo do UAV e processa cada frame individualmente através do modelo CLIP ViT-B/32 pré-treinado. Para cada imagem é gerado um embedding visual de características semânticas e, posteriormente, é calculada a média de todos os embeddings das imagens da janela temporal, produzindo um embedding compacto de 512 dimensões representativo do contexto visual observado pelo UAV. **Nota:** Nesta simulação temos o envio de dados da camara 1 vez por segundo e é quando estes dados são recebidos na OBC unit que acaba por haver a agregação e os processos seguintes a ela.

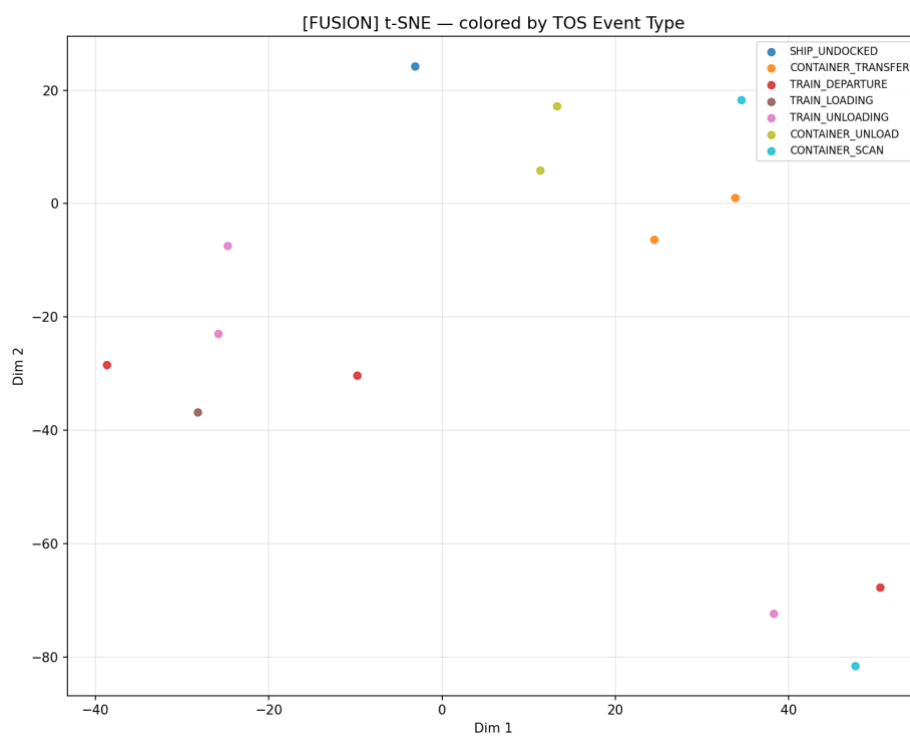
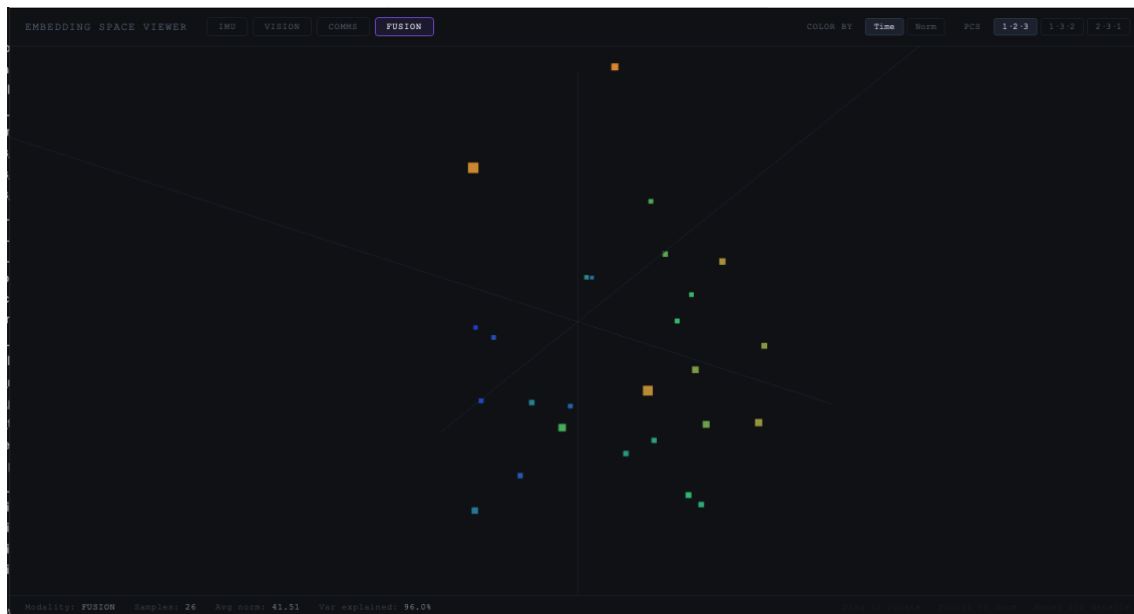
4. Antes de serem publicados os dados são agregados num numpy array de dimensão (768,). Estes dados são publicados no tópico (`world.state.tokens`) incluídos no seguinte payload:

message_type	Type of transmitted message [uav_embedding]
uav_id	UAV identifier
timestamp	Payload generation timestamp [s]
embedding	UAV fused embedding vector [numpy_arr(768,)]
metadata.has_imu	Presence or absence of IMU data [bool]
metadata.has_camera	Presence or absence of Vision/cam data [bool]
metadata.has_comms	Presence or absence of comms data [bool]

Nota: Os dados ainda são todos publicados no mesmo tópico, sendo que no final talvez faça mais sentido, em termos de escalabilidade e organização ter tópicos diferentes para receber dados dos diferentes agentes/componentes da arquitetura.

Nesta primeira versão só se está a enviar esta mensagem quando há dados das 3 fontes diferentes, no entanto o payload ficou já preparado para haver falta de dados de alguma das fontes.





Este exemplo ficou melhor representado pelo algoritmo t-SNE (t-Distributed Stochastic Neighbor Embedding), uma vez que o PCA capturou uma variabilidade muito dependente de uma dimensão dos dados (possivelmente o tipo de evento do TOS)

Apontamentos:

- O streaming + real-time processing de batches liga muito bem com a fusão de dados multimodais para manter consistência de janelas temporais entre as diferentes fontes